

强化学习概述（二）：Policy Gradient

Return Estimation, Baseline, On-policy, Off-policy, PPO, and Exploration



基于李宏毅老师《机器学习 2025》课程内容整理

2026 年 6 月 9 日

视频信息

频道：李宏毅深度学习课堂 (Bilibili)

时长：约 41 分 13 秒

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=47>

目录

1 从 P46 留下的问题开始	3
1.1 本章小结	3
2 Version 0: 即时 reward 当作动作评价	4
2.1 reward delay 与短视问题	5
2.2 本章小结	7
3 Version 1 与 Version 2: 累计与折扣累计 reward	7
3.1 折扣因子	8
3.2 本章小结	9
4 Version 3: 减 baseline, 得到相对好坏	9
4.1 本章小结	10
5 Policy Gradient 的训练流程	11
5.1 为什么只能更新一次	11
5.2 旧 trajectory 为什么不能随便复用	12
5.3 本章小结	13
6 On-policy、Off-policy 与 PPO	13
6.1 PPO 的位置	15
6.2 本章小结	16
7 Exploration: 为什么 actor 要有随机性	16
7.1 PPO 在控制任务中的例子	17
7.2 走向 Actor-Critic	19
7.3 本章小结	20
8 总结与延伸	20

1 从 P46 留下的问题开始

上一讲把 policy gradient 的直觉写成加权 cross-entropy: 如果在 state s_n 下采取 action a_n 是好事, 就给它正权重; 如果是坏事, 就给它负权重。训练 actor 的目标可以写成

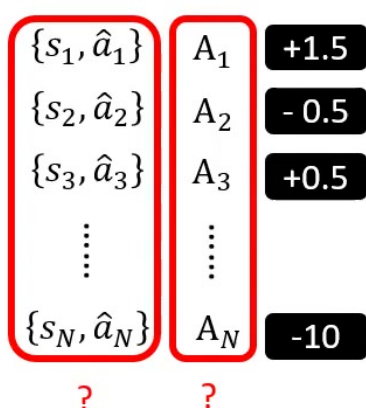
$$L(\theta) = \sum_{n=1}^N A_n e_n = - \sum_{n=1}^N A_n \log \pi_{\theta}(a_n | s_n),$$

其中 $e_n = -\log \pi_{\theta}(a_n | s_n)$ 。问题是: 这些 (s_n, a_n) 从哪里来? 权重 A_n 又怎么知道?

中文字幕 白宇工作室 bilibili

How to control your actor

Training Data



$$L = \sum A_n e_n$$
$$\theta^* = \arg \min_{\theta} L$$

图 1: P46 结尾留下两个问号: 训练 actor 的 state-action pair 和每个 pair 的权重 A_n 从哪里来? ¹

P47 就是在回答这两个问题。最自然的答案是: 让 actor 自己去和环境互动, 收集许多 episode。每一次互动都会产生 state、action 和 reward。state-action pair 直接来自 trajectory; 而 A_n 则要从 reward 中估计。接下来课程用 Version 0、1、2、3 逐步改进 A_n 的定义。

policy gradient 的核心任务

policy gradient 要把“整条 trajectory 的好坏”转换成“每个动作该被鼓励还是抑制”的权重。这个权重就是本节反复出现的 A_n , 它可以理解为某个动作相对于平均水平的好坏程度。

1.1 本章小结

P47 的主线不是重新定义 actor, 而是补上 P46 中缺失的训练信号来源: 用当前 actor 收集 trajectory, 再从 reward 中构造 A_n , 最后用加权 cross-entropy 更新 actor。

¹ 视频画面时间区间: 00:00:00-00:00:15。

2 Version 0: 即时 reward 当作动作评价

最简单的做法是收集许多 episode，把每一步的 state-action pair 作为训练资料：

$$\mathcal{D} = \{(s_1, a_1), (s_2, a_2), \dots, (s_N, a_N)\}.$$

然后直接把该步得到的即时 reward 当作权重：

$$A_t = r_t.$$

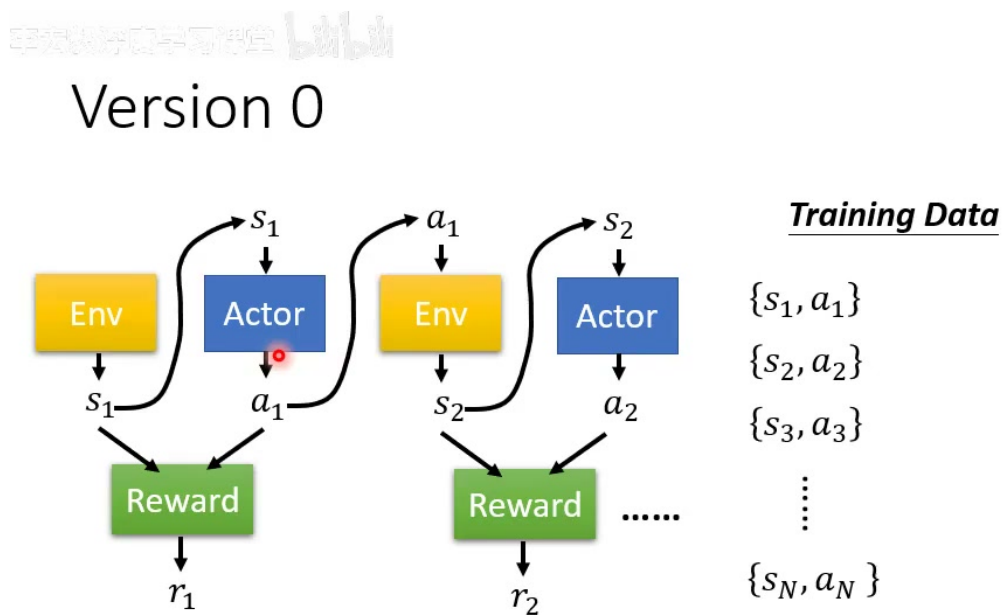


图 2: Version 0: actor 与环境互动，收集许多 (s_t, a_t) pair 作为训练资料。²

²视频画面时间区间：00:04:15–00:04:30。

Version 0

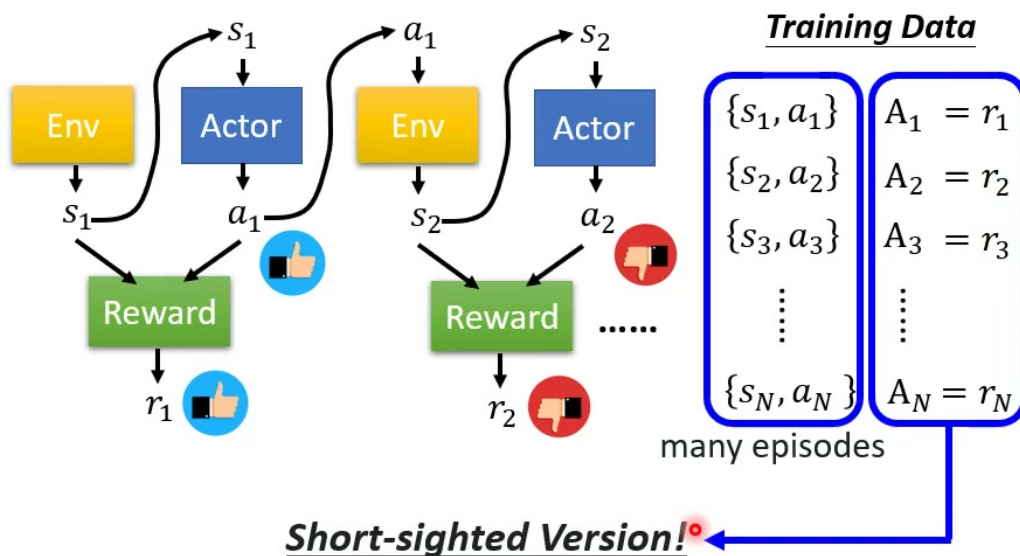


图 3: Version 0 把每一步的 A_t 直接设为即时 reward r_t , 因此是一个 short-sighted version。³

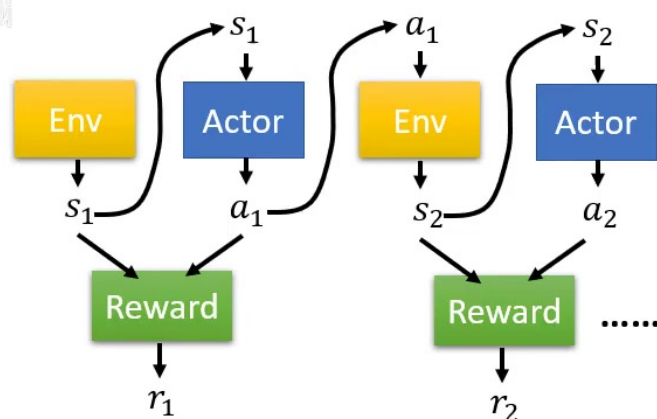
这个版本很直观：某一步得到正 reward，就鼓励这一步动作；某一步得到负 reward，就抑制这一步动作。但它的问题也很明显：动作的好坏不一定在同一时刻显现。很多动作是为未来铺路，短期看没有 reward，长期却很重要。

2.1 reward delay 与短视问题

在 RL 中，一个 action 会影响之后的 observation，也会影响之后能得到的 reward。若只看 r_t ，actor 会变得短视。课程用 Space Invader 说明：如果只有“开火击中外星人”那一步有正 reward，而向右移动暂时没有分数，那么 Version 0 会学到“只有开火是好动作”，却学不到先移动再开火的策略。

³视频画面时间区间：00:07:15–00:07:30。

Version 0

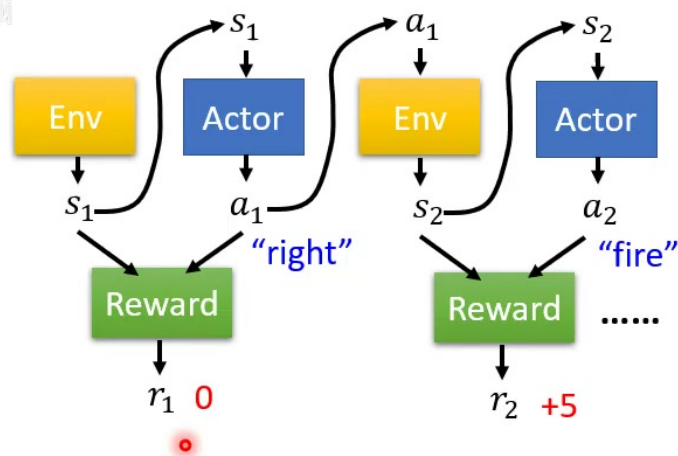


- An action affects the subsequent observations and thus subsequent rewards.
- *Reward delay*: Actor has to sacrifice immediate reward to gain more long-term reward.

图 4: reward delay: 动作会影响后续观察与后续 reward, actor 有时必须牺牲即时 reward 来换取长期 reward。⁴

⁴视频画面时间区间: 00:08:30-00:08:45。

Version 0



- An action affects the subsequent observations and thus subsequent rewards.
- *Reward delay*: Actor has to sacrifice immediate reward to gain more long-term reward.
- In space invader, only “fire” yields positive reward, so version 0 will learn an actor that always “fire”.

图 5: Space Invader 中，若只把即时 reward 当评价，actor 可能只学会不断开火，而不会学习先移动到合适位置。⁵

即时 reward 不是动作价值

r_t 只描述当前动作后的即时反馈，不等于 a_t 对整局结果的贡献。若任务存在延迟回报，只用 r_t 会造成错误 credit assignment。

2.2 本章小结

Version 0 的训练资料来源很自然，但 $A_t = r_t$ 太短视。它忽略了动作对未来状态和未来 reward 的影响，因此无法处理 reward delay。下一步要把未来 reward 也纳入动作评价。

3 Version 1 与 Version 2: 累计与折扣累计 reward

Version 1 把一个动作之后所有 future reward 加起来，作为该动作的评价。对第 t 步动作，定义 return-to-go:

$$G_t = \sum_{n=t}^N r_n.$$

然后令

$$A_t = G_t.$$

这比即时 reward 合理：若 a_t 暂时没有 reward，却让未来拿到高分，它仍然会得到正评价。

⁵视频画面时间区间：00:09:45–00:10:00。

Version 1

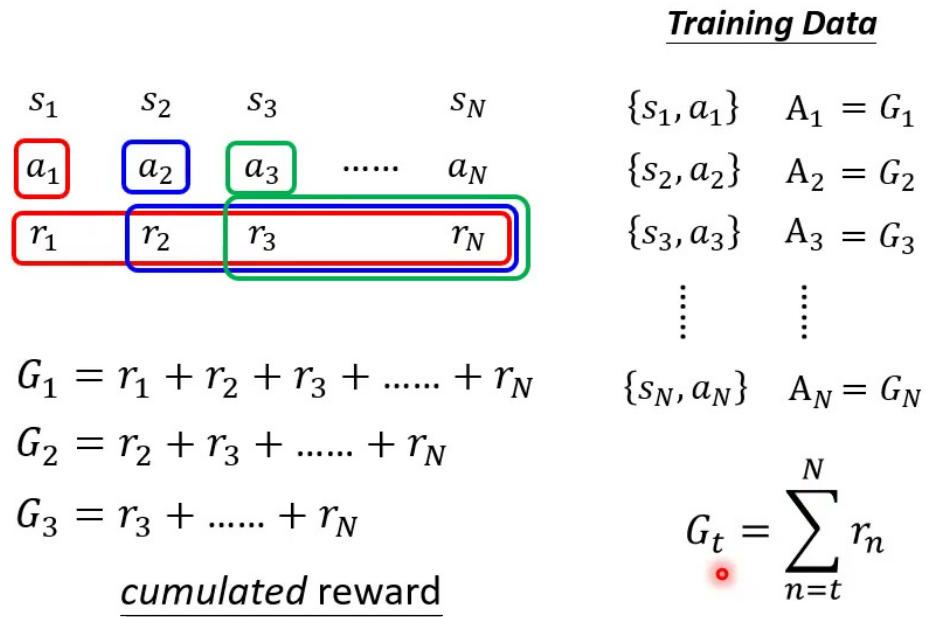


图 6: Version 1: 用从当前时间点到 episode 结束的累计 reward G_t 评价动作。⁶

不过, Version 1 又有另一个问题: 很遥远的 reward 也被完全归功于早期动作。若 episode 很长, 把 r_N 全额算到 a_1 头上, 未必合理。早期动作当然可能影响未来, 但影响通常会随着时间变远而减弱。

3.1 折扣因子

Version 2 引入 discount factor $\gamma < 1$, 让更远的 reward 权重更小:

$$G'_t = \sum_{n=t}^N \gamma^{n-t} r_n = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

然后令

$$A_t = G'_t.$$

⁶视频画面时间区间: 00:12:15-00:12:30。

Version 2

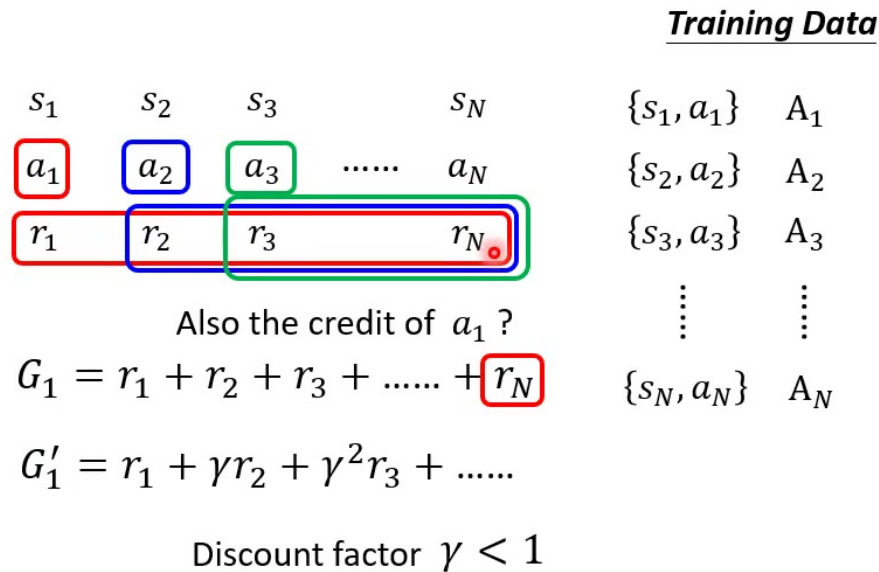


图 7: Version 2: 用折扣因子 $\gamma < 1$ 降低远期 reward 的权重，避免把太遥远的结果完全归功于当前动作。⁷

折扣因子的作用有两层。第一，它符合“越近的结果越可能与当前动作有关”的直觉。第二，它让数值更稳定，尤其在 episode 很长甚至无限长时，折扣和可以保持有限。常见 RL 文献中， γ 也表达 agent 对未来的重视程度。

return、reward、advantage 的关系

reward 是单步反馈，return 是从某步开始累积的未来反馈，advantage 则通常表示某个动作比基准预期好多少。本节的 A_t 先从 reward、return 开始，最后通过减 baseline 接近 advantage 的形式。

3.2 本章小结

Version 1 用未来累计 reward 解决短视问题，但可能过度归功于远期结果。Version 2 加入折扣因子，让未来 reward 的贡献随时间衰减。这已经比即时 reward 合理得多，但仍然缺少“相对好坏”的概念。

4 Version 3: 减 baseline，得到相对好坏

课程接着指出：reward 的好坏是相对的。如果一次考试你拿 60 分，但全班平均 50 分，那 60 分不错；如果全班平均 80 分，那 60 分就很差。RL 中也是一样，若某个游戏里每个动作都得到正分，只看正 reward 会让所有动作都被鼓励，actor 不知道哪些动作其实只是“低于平均”。

⁷ 视频画面时间区间：00:14:45–00:15:00。

Version 3 因此引入 baseline b :

$$A_t = G'_t - b.$$

这样 A_t 可以有正有负。若 G'_t 高于 baseline，动作被鼓励；若低于 baseline，动作被抑制。



Version 3

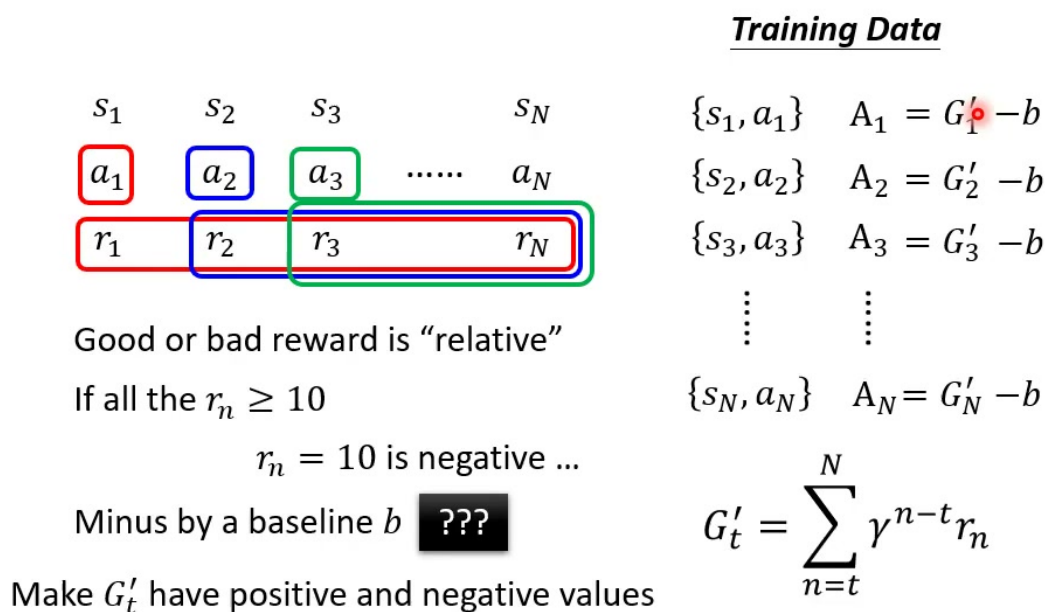


图 8: Version 3: 从折扣累计 reward 中减去 baseline b ，让 A_t 同时有正值和负值。⁸

在标准 policy gradient 中，这个 A_t 往往被称为 advantage。最理想的形式是

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t),$$

其中 $Q^\pi(s_t, a_t)$ 表示在 s_t 做 a_t 后的预期 return， $V^\pi(s_t)$ 表示在 s_t 按当前 policy 正常行动的平均预期 return。课程在这里先不展开 Q 和 V ，而是用 baseline 的直觉说明：动作好不好要看它是否比通常情况更好。

baseline 的两个作用

baseline 不改变“相对鼓励哪个动作”的核心方向，却能让权重有正有负，并降低梯度估计方差。直觉上，它把“拿到多少分”变成“比预期多拿多少分”。

4.1 本章小结

Version 3 把 A_t 从绝对 return 改成相对 return。减 baseline 后，actor 不再盲目鼓励所有正 reward 动作，而是鼓励高于基准的动作、抑制低于基准的动作。这是 policy gradient 稳定训练的重要思想。

⁸视频画面时间区间：00:22:15–00:22:30。

5 Policy Gradient 的训练流程

把前面的 A_t 估计放进训练流程，就得到本节的 policy gradient 框架。先初始化 actor 参数 θ^0 。在第 i 次训练迭代中，用当前 actor θ^{i-1} 与环境互动，收集 trajectory；计算每个 state-action pair 的 A_n ；构造 loss：

$$L(\theta) = \sum_{n=1}^N A_n e_n = - \sum_{n=1}^N A_n \log \pi_{\theta}(a_n | s_n);$$

再做一次梯度更新：

$$\theta^i \leftarrow \theta^{i-1} - \eta \nabla_{\theta} L(\theta).$$



Policy Gradient

- Initialize actor network parameters θ^0
- For training iteration $i = 1$ to T
 - Using actor θ^{i-1} to interact
 - Obtain data $\{s_1, a_1\}, \{s_2, a_2\}, \dots, \{s_N, a_N\}$
 - Compute $A_1, A_2, \dots, A_N$
 - Compute loss L
 - $\theta^i \leftarrow \theta^{i-1} - \eta \nabla L$

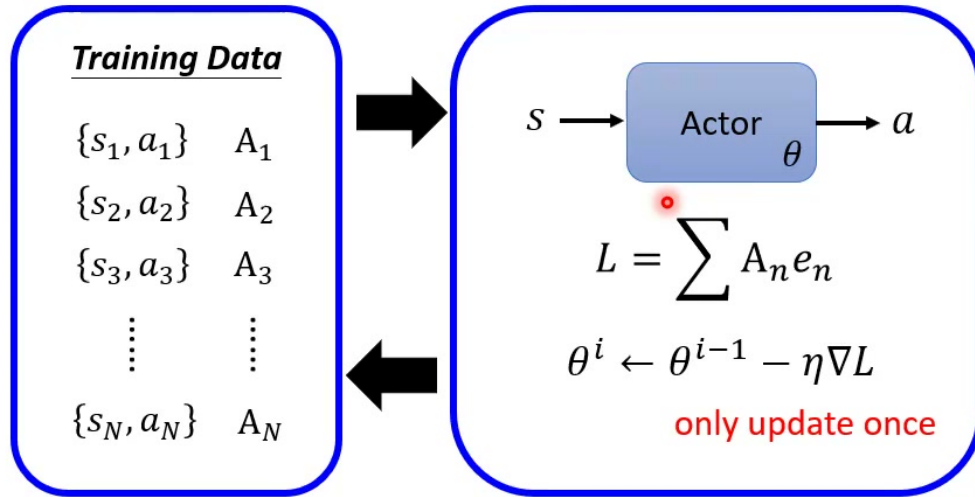
图 9: policy gradient 流程：用 actor 与环境互动收集资料，计算 A_n ，再用加权 loss 更新参数。⁹

5.1 为什么只能更新一次

课程特别强调一个与普通 supervised learning 不同的地方：在 supervised learning 中，训练资料通常先收集好，然后可以反复训练很多 epoch；但在 on-policy policy gradient 中，用 θ^{i-1} 收集到的数据，原则上只适合用来更新 θ^{i-1} 一次。更新成 θ^i 后，新的 actor 会产生不同的 trajectory，旧资料可能不再代表新 actor 的行为分布。

⁹ 视频画面时间区间：00:23:30–00:23:45。

Policy Gradient



Each time you update the model parameters, you need to collect the whole training set again.

图 10: on-policy PG 中，每次更新参数后通常需要重新收集整批训练资料；旧资料只用于一次更新。¹⁰

这就是 RL 训练很花时间的一个原因。每次参数更新都要与环境互动、重新采样 trajectory、重新计算 A_n 。如果环境是真实机器人或真实用户系统，收集资料还可能非常昂贵。

5.2 旧 trajectory 为什么不能随便复用

假设旧 actor θ^{i-1} 在前几步和新 actor θ^i 做了同样动作，后面仍然可能走向完全不同的 trajectory。旧资料中某个动作后续得到高 reward，不代表新 actor 采取同一前缀动作后也会看到相同后续状态。因为 policy 改了，未来动作分布也改了。

¹⁰ 视频画面时间区间：00:26:00–00:26:15。

Policy Gradient

- Initialize actor network parameters θ^0
- For training iteration $i = 1$ to T
 - Using actor θ^{i-1} to interact
 - Obtain data $\{s_1, a_1\}, \{s_2, a_2\}, \dots, \{s_N, a_N\}$
 - Compute $A_1, A_2, \dots, A_N$
 - Compute loss L
 - $\theta^i \leftarrow \theta^{i-1} - \eta \nabla L$

Trajectory of θ^{i-1}

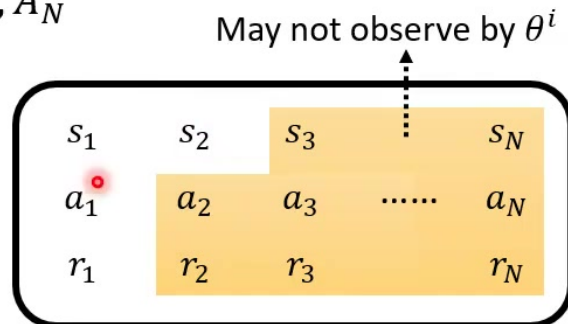


图 11: 旧 actor 收集到的 trajectory 后半段, 新的 actor 不一定会观察到; 这就是旧数据难以直接复用的原因。¹¹

on-policy 的样本效率问题

on-policy 方法通常更直接、更符合推导假设, 但样本效率较低。因为每次 policy 更新后, 旧数据就变得不完全匹配, 需要重新与环境互动采样。

5.3 本章小结

policy gradient 的训练流程可以看成“采样、评估、更新、再采样”。它与监督学习最大的流程差异是数据收集在训练循环内部, 而不是一次性固定在循环外部。这也引出 on-policy 与 off-policy 的区别。

6 On-policy、Off-policy 与 PPO

on-policy 指的是: 要训练的 actor 和用来与环境互动、收集资料的 actor 是同一个。off-policy 则允许两者不同: 一个 actor 负责收集经验, 另一个 actor 利用这些经验学习。off-policy 的好处很直接: 可以复用旧资料, 不必每次更新都重新收集完整 trajectory。

¹¹视频画面时间区间: 00:29:45-00:30:00。

On-policy v.s. Off-policy

- The **actor to train** and the **actor for interacting** is the same. → On-policy •
- Can the **actor to train** and the **actor for interacting** be different? → Off-policy

图 12: on-policy 与 off-policy 的定义: 训练用 actor 和互动收集资料的 actor 是否相同。¹²

¹²视频画面时间区间: 00:31:00-00:31:15。

On-policy v.s. Off-policy

- The **actor to train** and the **actor for interacting** is the same. → On-policy
- Can the **actor to train** and the **actor for interacting** be different? → Off-policy

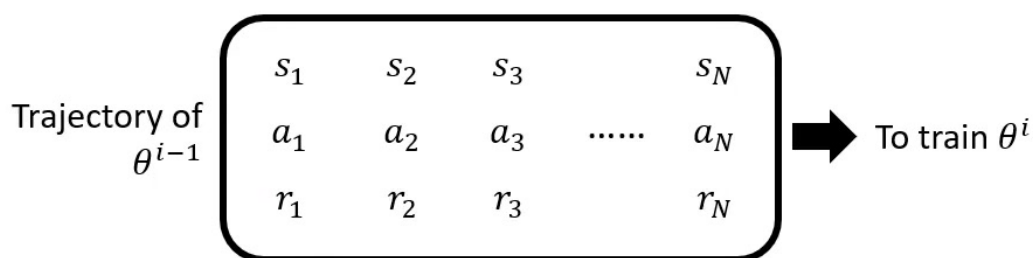


图 13: off-policy 试图用旧 actor 收集的 trajectory 来训练新 actor，以减少重复采样成本。¹³

但 off-policy 也有困难：若训练 actor 与互动 actor 差太多，旧经验对新 actor 的意义会打折扣。某个动作对旧 actor 是好经验，不代表新 actor 在相同状态下会延续到同样未来。严格算法需要处理 distribution mismatch，例如 importance sampling、trust region 或 clipped objective 等。

6.1 PPO 的位置

课程提到 Proximal Policy Optimization，简称 PPO，是今天常用的 off-policy / 近似 off-policy 思路之一。PPO 的核心精神是：训练 actor 要知道自己和收集资料的 actor 差多少，不要一次更新得太远。若新旧 policy 差异过大，旧数据就不可靠。

¹³ 视频画面时间区间：00:32:15–00:32:30。

Off-policy → Proximal Policy Optimization (PPO)

- The **actor to train** has to know its difference from the **actor to interact**.

<https://disp.cc/b/115-bLHe>

图 14: PPO 的直觉: 要训练的 actor 必须知道自己与收集资料的 actor 有多不同, 并限制更新幅度。¹⁴

虽然课程没有展开 PPO 推导, 但可以用一行公式理解其 clipped objective 的味道。设旧 policy 为 $\pi_{\theta_{\text{old}}}$, 新 policy 为 π_{θ} , 概率比为

$$\rho_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}.$$

PPO 会避免 ρ_t 偏离 1 太远, 防止新 policy 对旧 trajectory 做过度解释。

为什么 PPO 常见

PPO 通常比最朴素 policy gradient 稳定, 又比更复杂的 trust-region 方法实现简单, 因此在许多连续控制和模拟环境中成为常用 baseline。课程后面的 DeepMind 与 OpenAI 例子都与 PPO 相关。

6.2 本章小结

on-policy 方法数据匹配但耗样本; off-policy 方法想复用旧数据, 但必须处理旧 actor 与新 actor 的差异。PPO 的核心动机就是在可复用数据和不过度偏离旧 policy 之间取得平衡。

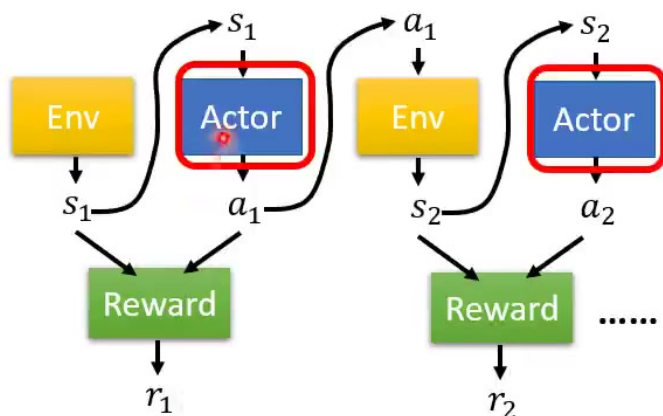
7 Exploration: 为什么 actor 要有随机性

课程最后回到一个实践问题: 数据从哪里来? 若 actor 只会执行当前认为最好的动作, 它可能永远不会尝试其他动作, 也就永远不知道其他动作是否更好。例如 Space Invader 中, 如果 actor 从不

¹⁴ 视频画面时间区间: 00:33:30–00:33:45。

尝试开火，就不会知道开火能带来 reward。为了收集有价值的训练资料，actor 在 data collection 时必须有随机性。

Collection Training Data: Exploration



The actor needs to have randomness during data collection.

A major reason why we sample actions. 😊

图 15: exploration: actor 在收集资料时需要随机性; 这也是 policy 输出动作分布、从中采样的重要理由。¹⁵

常见 exploration 方法包括:

- 按 $\pi_{\theta}(a | s)$ 采样动作，而不是总取最大概率动作;
- 提高输出分布的 entropy，让动作概率更分散;
- 在参数或动作上加入噪声;
- 在值函数方法中使用 ϵ -greedy 等探索策略。

本节以 policy gradient 为主，所以重点是：采样动作不是实现细节，而是 RL 学习机制的一部分。没有探索，训练资料会太窄；资料太窄，actor 就很难发现更好的策略。

7.1 PPO 在控制任务中的例子

课程展示了 DeepMind 与 OpenAI 的 PPO 控制任务视频。例如让虚拟人学习走路、跑步、跨越障碍或在球场环境中运动。它们看起来有时很滑稽，但核心是同一个 RL 问题：actor 通过与模拟环境互动，逐渐学会能带来更高 reward 的动作序列。

¹⁵ 视频画面时间区间：00:36:00–00:36:15。

DeepMind - PPO

<https://youtu.be/gn4nRCC9TwQ>



图 16: DeepMind PPO 控制任务示例：虚拟角色通过 reward 学习行走与跨越地形。¹⁶

¹⁶视频画面时间区间：00:38:30–00:38:45。



图 17: OpenAI PPO 控制任务示例：在模拟环境中训练连续控制策略，展示 policy gradient 方法的应用场景。¹⁷

这些例子也呼应了本讲标题中的“修课心情”：RL 训练常常充满不稳定、反复尝试、结果看起来怪但又逐渐变好。算法层面是 exploration 与 policy update，实践层面则是大量调参、重跑和观察曲线。

7.2 走向 Actor-Critic

P47 最后回到大纲，下一部分是 Actor-Critic。它会继续处理一个核心问题：如何更好地估计 A_t ？单纯用 sampled return 方差很大，若能学习一个 critic 来估计 value 或 advantage，就可以让 actor 的更新更稳定。

¹⁷ 视频画面时间区间：00:39:45–00:40:00。

Outline



图 18: P47 结束后，大纲进入 Actor-Critic；它会用 critic 来改进 actor 更新所需的价值估计。¹⁸

7.3 本章小结

exploration 让 actor 有机会发现未知动作的价值；PPO 等方法则在 policy 更新稳定性和资料复用之间折中。下一步 Actor-Critic 会把“如何估计 A_t ”这个问题继续推进，用 critic 降低 policy gradient 的方差。

8 总结与延伸

这一讲把 P46 的直觉补成了可执行的 policy gradient 流程。首先，让 actor 与环境互动，收集 trajectory；其次，从 reward 中为每个 state-action pair 构造权重 A_t ；最后，用

$$L(\theta) = - \sum_t A_t \log \pi_\theta(a_t | s_t)$$

更新 actor。核心变化在于 A_t 的定义：Version 0 用即时 reward，太短视；Version 1 用未来累计 reward，解决 reward delay；Version 2 加入 discount factor，降低远期 reward 的影响；Version 3 减 baseline，把绝对分数变成相对好坏。

整节课最重要的概念压缩如下：

return-to-go $G_t = \sum_{n=t}^N r_n$ ，用当前之后的总 reward 评估动作。

discounted return $G'_t = \sum_{n=t}^N \gamma^{n-t} r_n$ ，让远期 reward 衰减。

baseline / advantage $A_t = G'_t - b$ ，鼓励高于基准的动作，抑制低于基准的动作。

on-policy 用当前 actor 收集资料并训练当前 actor，数据匹配但耗样本。

¹⁸ 视频画面时间区间：00:40:45–00:41:00。

off-policy / PPO 尝试复用旧资料，但要控制新旧 actor 的差异。

exploration actor 必须随机尝试动作，否则无法知道未尝试动作是否更好。

从 P46 到 P47 的完整闭环

P46 告诉我们可以用加权 cross-entropy 控制 actor；P47 告诉我们权重从哪里来：它来自 trajectory 中的 future reward，并通过 discount 与 baseline 修正。这样，长期 reward 被转换成了局部动作概率的更新方向。

后续 Actor-Critic 会继续修正 policy gradient 的高方差问题。直觉上，actor 负责行动，critic 负责评价；critic 若能估计某个 state 或 action 的价值，就能给 actor 更可靠的 A_t 。这也是强化学习从“靠采样回报更新”走向更稳定算法的重要一步。